

Optimization Algorithms In Machine Learning

Prepared By
Hager Maged

Introduction

Optimization is the process of adjusting a model's internal parameters, most commonly its weights and biases, so that a chosen loss function is minimized. Training a machine learning model is, at its core, an optimization problem: the learning algorithm repeatedly updates the parameters, searching for the values that make the model's predictions as close as possible to the true values.

Optimizers are important during training for several reasons:

- 1. Convergence Speed:** A good optimizer reaches a low-loss solution in fewer iterations, which saves computation time and resources, especially for large models.
- 2. Model Performance:** The choice of optimizer affects how well the model generalizes to unseen data. A poorly chosen optimizer can leave the model stuck with a high loss or overfit to the training data.
- 3. Avoiding Local Minima and Saddle Points:** Loss surfaces in deep learning are rarely simple bowls. Optimizers differ in how well they escape flat regions, saddle points, and shallow local minima.
- 4. Computational Efficiency:** Some optimizers are designed to work well with large datasets or high-dimensional parameter spaces, making training feasible in practice.

The loss function and the optimization algorithm are tightly connected. The loss function defines the surface over which the optimizer searches, and its gradient (the direction of steepest increase) tells the optimizer which way to move. The optimizer then decides how large a step to take and how to use that gradient information, whether directly, as in Gradient Descent, or combined with historical information, as in Adam. In short, the loss function defines what to minimize, while the optimizer defines how to minimize it.

Optimizer Explanation

1. Gradient Descent (GD)

A. Definition

Gradient Descent is the most fundamental optimization algorithm used to minimize a loss function. It works by iteratively moving the model's parameters in the direction opposite to the gradient of the loss with respect to those parameters. Since the gradient points toward the direction of steepest increase, moving against it decreases the loss.

$$\theta = \theta - \eta \nabla J(\theta)$$

where θ represents the model parameters, η is the learning rate, and $\nabla J(\theta)$ is the gradient of the loss function computed using the entire training dataset.

B. Working Principle

At each iteration, GD computes the gradient of the loss function using the whole training dataset, then updates all parameters simultaneously by taking a step in the negative gradient direction, scaled by the learning rate. This process repeats until the gradient becomes very small or the loss stops improving.

C. Advantages

- ✓ Produces a stable and smooth convergence path since the full dataset is used at every step.
- ✓ Guaranteed to converge to the global minimum for convex loss functions.
- ✓ Simple to understand and implement, and forms the theoretical basis for the other optimizers.

D. Disadvantages

- ✗ Computationally expensive for large datasets, since a full pass over the data is required for a single update.
- ✗ Slow in practice, especially for high-dimensional problems.
- ✗ Requires the entire dataset to fit in memory.

E. Typical Applications

GD is mainly used for small-scale problems, such as linear or logistic regression on small datasets, and for theoretical analysis and teaching, where the dataset is small enough to fit comfortably in memory.

2. Stochastic Gradient Descent (SGD)

A. Definition

Stochastic Gradient Descent is a variant of GD that updates the parameters using the gradient computed from a single randomly chosen training example (or a small mini-batch) instead of the entire dataset.

$$\theta = \theta - \eta \nabla J(\theta; x_i, y_i)$$

where (x_i, y_i) is a single training example randomly selected at each step.

B. Working Principle

At each step, SGD randomly selects one training example (or mini-batch), computes the gradient of the loss for that sample only, and updates the parameters immediately. Because every update is based on a different, small subset of the data, the path toward the minimum is noisy and zig-zagging rather than smooth, as illustrated in Figure 1, but on average it still moves toward the minimum, and each individual update is much faster to compute.

C. Advantages

- ✓ Much faster per update, since only one sample or a small batch is used.
- ✓ The noise in the updates can help the model escape shallow local minima and saddle points.
- ✓ Well suited to online learning and streaming data.
- ✓ Requires far less memory than GD, since the full dataset is never loaded at once.

D. Disadvantages

- ✗ Noisy updates cause the loss to fluctuate instead of decreasing smoothly.
- ✗ May overshoot the minimum, often requiring a learning-rate schedule.
- ✗ Convergence is less stable than GD, making it harder to know when to stop training.

E. Typical Applications

SGD (and its mini-batch variant) is the basis of most large-scale deep learning training, and is also used in online learning systems and situations where data arrives continuously as a stream.

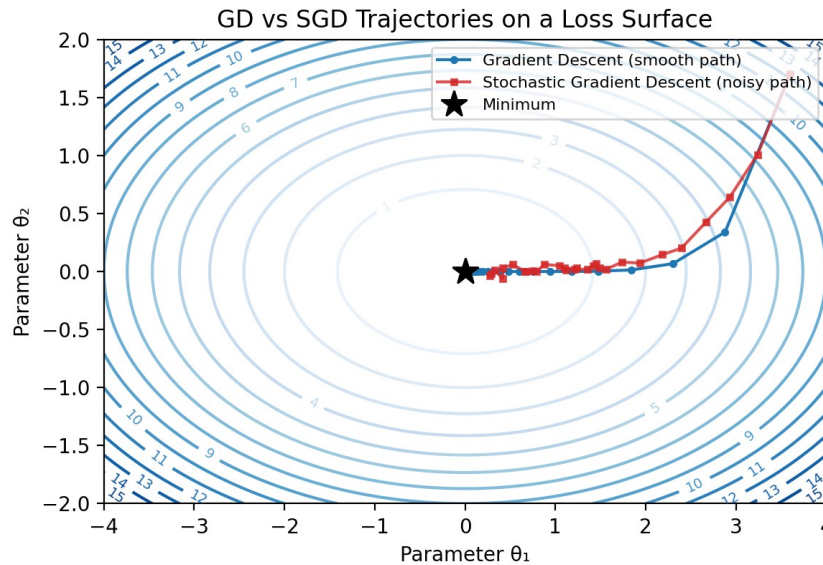


Figure 1: GD vs SGD Trajectories on a Loss Surface

As shown in Figure 1, Gradient Descent follows a smooth, direct path toward the minimum, while Stochastic Gradient Descent follows a noisier, zig-zagging path because each step is based on a different sample of the data rather than the full dataset.

3. AdaGrad (Adaptive Gradient Algorithm)

A. Definition

AdaGrad adapts the learning rate for each parameter individually, based on the historical sum of the squared gradients of that parameter. Parameters that receive large or frequent gradients get progressively smaller updates, while parameters with small or infrequent gradients keep relatively larger updates.

$$G_t = G_{t-1} + g_t^2$$

$$\theta = \theta - (\eta / \sqrt{G_t + \epsilon}) g_t$$

where G_t is the accumulated sum of squared gradients up to time t , g_t is the gradient at time t , and ϵ is a small constant added to avoid division by zero.

B. Working Principle

AdaGrad keeps a running sum of squared gradients for every parameter separately. This sum grows continuously as training progresses, so the effective learning rate for each parameter shrinks over time, more quickly for parameters that have received large gradients. In effect, each parameter ends up with its own adaptive learning rate.

C. Advantages

- ✓ Performs well on sparse data, such as text or NLP features, since rare features receive comparatively larger updates.

- ✓ Removes the need to manually tune a separate learning rate for every parameter.
- ✓ Well suited to convex optimization problems.

D. Disadvantages

- ✗ The accumulated sum of squared gradients grows continuously, causing the learning rate to shrink and eventually approach zero, which can stop learning prematurely.
- ✗ Not well suited to deep, non-convex networks trained over many iterations.

E. Typical Applications

AdaGrad is commonly used for NLP tasks involving sparse gradients, such as training word embeddings, and for other problems with sparse features, such as recommender systems.

4. RMSProp (Root Mean Square Propagation)

A. Definition

RMSProp addresses AdaGrad's shrinking learning-rate problem by replacing the accumulated sum of all past squared gradients with an exponentially decaying moving average of squared gradients.

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$$

$$\theta = \theta - (\eta / \sqrt{E[g^2]_t + \epsilon}) g_t$$

where β is a decay rate, typically around 0.9, that controls how quickly older gradients are forgotten.

B. Working Principle

Instead of summing all past squared gradients, RMSProp maintains a moving average that gives more weight to recent gradients and gradually forgets older ones. This prevents the denominator in the update rule from growing without bound, keeping the effective learning rate useful throughout training rather than vanishing.

C. Advantages

- ✓ Solves AdaGrad's diminishing learning-rate problem.
- ✓ Performs well on non-stationary objectives, such as those found in recurrent neural networks.
- ✓ Works well in online and mini-batch training settings.

D. Disadvantages

- ✗ Introduces an additional hyperparameter, the decay rate β , that needs to be tuned.
- ✗ Still sensitive to the choice of the base learning rate.
- ✗ Lacks an explicit momentum term, which can make convergence slower in some cases, an issue later addressed by Adam.

E. Typical Applications

RMSProp is widely used for training recurrent neural networks (RNNs) and LSTMs, and more generally for deep networks trained on noisy or non-stationary objectives.

5. Adam (Adaptive Moment Estimation)

A. Definition

Adam combines the ideas of momentum, tracking a moving average of past gradients, with the adaptive per-parameter learning rates of RMSProp. It is currently the most widely used optimizer in deep learning.

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= m_t / (1 - \beta_1^t) \quad \text{and} \quad \hat{v}_t = v_t / (1 - \beta_2^t) \\ \theta &= \theta - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)\end{aligned}$$

where m_t and v_t are moving averages of the gradient and the squared gradient respectively (the first and second moments), and $\hat{m}_t$, $\hat{v}_t$ are their bias-corrected versions, needed because both moments are initialized at zero and are therefore biased toward zero early in training.

B. Working Principle

Adam maintains two moving averages for every parameter: the first moment, which behaves like momentum and smooths the direction of movement, and the second moment, which behaves like RMSProp and scales the step size for each parameter individually. The bias-correction step compensates for the fact that both moving averages start at zero. The result combines the directional stability of momentum with the adaptive step sizes of RMSProp.

C. Advantages

- ✓ Combines the benefits of momentum and adaptive learning rates, giving fast and stable convergence.
- ✓ Performs well across a wide range of problems with little manual hyperparameter tuning.
- ✓ Handles sparse gradients and noisy objectives effectively.
- ✓ Used as the default optimizer in most deep learning frameworks.

D. Disadvantages

- ✗ Requires more memory, since it stores both the first and second moment estimates for every parameter.
- ✗ Can sometimes generalize slightly worse than SGD with momentum on certain tasks.
- ✗ Has more hyperparameters (β_1 , β_2 , ϵ) to configure, though the default values usually work well.

E. Typical Applications

Adam is the default optimizer for training most modern deep learning models, including convolutional neural networks, transformers, and generative adversarial networks, and is used across computer vision, NLP, and reinforcement learning.

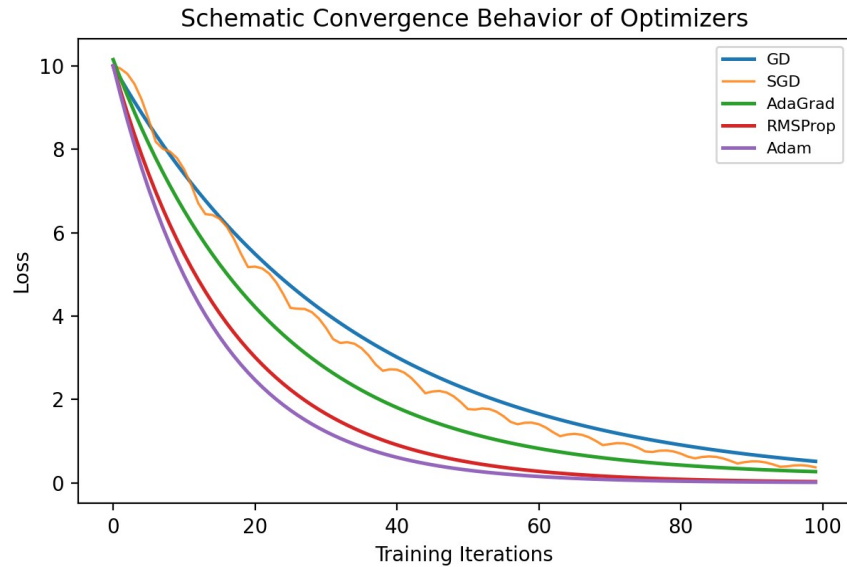


Figure 2: Schematic Convergence Behavior of Optimizers

As shown in Figure 2, adaptive optimizers such as RMSProp and Adam tend to reduce the loss faster and more smoothly than plain GD, while SGD converges quickly on average but with a visibly noisier curve due to its use of individual samples rather than the full dataset.

Comparison Table

Optimizer	Speed	Memory Usage	Learning Rate Adaptation	Main Advantage
GD	Slow (uses the full dataset per step)	High (gradients over the entire dataset)	None (fixed learning rate)	Stable, smooth convergence
SGD	Fast per update, but noisy overall	Low (one sample/mini-batch at a time)	None (fixed, though schedules are often added)	Efficient for large-scale and streaming data
AdaGrad	Moderate, slows down over time	Moderate (stores accumulated squared gradients)	Yes — per-parameter, monotonically decreasing	Effective for sparse features
RMSProp	Fast and stable	Moderate (stores a moving average per parameter)	Yes — per-parameter, based on a decaying average	Handles non-stationary objectives well
Adam	Fast and stable	Higher (stores two moment vectors per parameter)	Yes — per-parameter, momentum + adaptive scaling	Robust default choice with fast convergence

References

Ruder, S. (2016). An overview of gradient descent optimization algorithms. arXiv preprint arXiv:1609.04747.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR).

Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12, 2121–2159.

Hinton, G., Srivastava, N., & Swersky, K. *Neural Networks for Machine Learning*, Lecture 6e: RMSProp. Coursera / University of Toronto.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.